

Matrix

- Sparse matrix
- Dense matrix

Sparse Retrieval (Heuristic retrieval, syntactic retrieval) - BM25

* Retrieval for sparse representations (=contains many zeros)

- sparse vectors for documents
- **corpus** (a **set** of documents, document: a **bag** of terms) -> a sparse **matrix** = **term-document** matrix

- inverted index: {(term, posting list)}
 - {(against, (4:1, 7:3, 8:2, 9:3, ..), ..., (ago, (4:2, 9:3, ..), ..)}
 - {(against, (4:1/5, 7:3/4, 8:2/2, 9:3/10, ..), ..., (ago, (4:2/6, 9:3/10, ..), ..)}
- posting list: doc ID list

	d1	d2	d3	d4	d5	d6	d7	d8	d9	d10	...
against	0	0	0	1	0	0	3	2	3	0	
age	0	0	0	1	0	3	1	0	4	0	
agent	0	0	0	0	0	0	0	0	0	0	
ages	0	0	0	0	0	2	0	0	0	0	
ago	0	0	0	2	0	0	0	0	3	0	
agree	0	1	0	0	0	0	0	0	0	0	
ahead	0	0	0	1	0	0	0	0	0	0	
ain't	0	0	0	0	0	0	0	0	0	0	
air	0	0	0	0	0	0	0	0	0	0	
aka	0	0	0	1	0	0	0	0	0	0	
⋮											

- document: a sequence of keywords
- term (keyword)

Boolean retrieval (no ranking)

q: a set of keywords (k1, k2, ..., kn)=> column vectors

= $\text{getPostingList}(k1) \cap \text{getPostingList}(k2) \dots \cap \text{getPostingList}(kn)$

Ranking!!

{d1, d2, ..., dm} m>>n

d3, d10, d25

Q: samsung electronics

TF-IDF (TF * IDF)

TF: term frequency

IDF: inverse document frequency

document frequency for a keyword 'samsung' : # of documents containing the keyword 'samsung'

IDF - samsung, electronics

TF-IDF: representation, ranking (dot, cosine similarity)

BM25

Dense Retrieval (Learning-based retrieval, semantic retrieval) - Cross Encoder (one encoder), Bi-Encoder (dual encoders, two encoders)

- * Retrieval for dense representations (= no zeros)
 - dense vectors for documents
 - term (token)

{{apple, (d3, d10, d20, ...,)}}

Document -> a dense vector

Query -> a dense vector

Ranking -> cosine similarity, dot product

- $\text{sim}(q, d) \Rightarrow \text{Enc}(q, d)$: a similarity value
- $\text{sim}(q, d) \Rightarrow \text{Enc}_Q(q) \text{ dot (or cosine similarity) } \text{Enc}_D(d)$

Finetuning of BERT

- Training data
- Loss function

Fusion of BM25 and Dense retrieval (Hybrid retrieval)

Information retrieval

D: corpus

$\text{rank}(q, D)$ - ranking all documents in D: $1 \sim |D|$

IR metrics

Benchmark

- BM25 > Cross Encoder ?

Vector database

Top-k search over high-dimensional vectors

d-dimensional vector

History

- 1) d is small
B-tree, R-tree ($d \leq 8$), space-partitioning tree: kd-tree, octree, ...
- 2) d is medium-sized

Curse of dimensionality (차원의 저주)

- 3) d is high

=> solutions: 1) LSH, 2) clustering-based indexing (IVF: Inverted File Index), 3) graph-based indexing

Inverted index: { (term, posting list) }